📄 MASTER MULTI-PHASE AI CODE GENERATION PROMPT

Complete School Management System (Mobile + Backend + Database + Notifications)

🧠 ROLE

You are a senior full-stack software architect and engineer.

You MUST build a complete, production-ready School Management System with:

- **React Native (Android + iOS mobile app)**
- **Node.js (Backend server with Express.js)**
- **MySQL (Database)**
- **Firebase Cloud Messaging (Push notifications)**

❗ CRITICAL EXECUTION RULES

- **Implement ONLY ONE PHASE at a time**
- **DO NOT generate future phases**
- **DO NOT skip any requirement**
- **Generate complete working code**
- **Use production-level architecture**
- **Include:**
  - **Backend code**
  - **Mobile app code**
  - **Database schema (SQL)**
  - **API routes**
  - **Error handling**
  - **Validation**

🧩 PHASE 1 — PROJECT SETUP + CORE ARCHITECTURE

Objective

Initialize full system architecture.

Requirements

1. **React Native CLI project setup:**
    - **Navigation (Stack + Tab)**
    - **Folder structure:**

```
/src
 /components
 /screens
 /navigation
 /redux
 /services
 /utils
 /themes
```

2. **Backend setup:**
    - **Express.js server**
    - **MVC structure:**

```
/src
 /controllers
 /models
 /routes
 /middlewares
 /services
 /utils
 /config
```

3. **Database:**
    - **MySQL connection setup**
    - **Environment config**
4. **Core setup:**
    - **Axios API service**

- Global error handler
- Environment variables

🧩 PHASE 2 — AUTHENTICATION + ROLES + OTP + SESSION

Objective

Secure login system with session handling.

Requirements

1. **Authentication:**
    - Register
    - Login
    - Logout
2. **Login methods:**
    - Email + Password
    - Phone + Password
    - Username + Password
3. **Security:**
    - Password hashing (bcrypt)
    - JWT (access + refresh tokens)
4. **OTP System:**
    - Send OTP via email
    - Trigger ONLY:
        - First login of the day
        - New device login
    - Store OTP with expiry
    - Verify OTP API
5. **Session Logic:**

    - Maintain login session for whole day
    - Avoid repeated login
6. Roles:
    - Admin
    - Staff
    - Teacher
    - Student
    - Parent
7. Database tables:
    - users
    - roles
    - otp_verifications
    - sessions
8. React Native screens:
    - Login
    - Register
    - OTP verification

🧩 PHASE 3 — ADMIN MODULE + USER MANAGEMENT (RBAC)

Objective

Admin full control over system.

Requirements

1. Admin Dashboard:
    - Stats (students, fees, attendance)
2. User Management:
    - CRUD:

        - Students
        - Teachers
        - Staff
        - Parents
3. Role Assignment:
    - Assign roles dynamically
4. Account control:
    - Activate / deactivate users
5. Backend:
    - Protected APIs (Admin only)
    - RBAC middleware
6. React Native:
    - Admin dashboard UI
    - User list
    - Add/Edit forms

🧩 PHASE 4 — TEACHER MODULE + ACADEMIC SYSTEM

Objective

Academic data management.

Requirements

1. Teacher Features:
    - Upload:
        - Videos
        - PDFs
        - Images
    - Upload study material

    - Upload assignments
2. Assignment Logic:
    - Assign to:
        - Whole class
        - Individual student
3. Marks:
    - Upload marks
    - Update marks
4. Attendance:
    - Daily marking
5. Backend:
    - File uploads (multer)
    - Storage handling
6. Database tables:
    - assignments
    - marks
    - attendance
    - materials
7. React Native:
    - Teacher dashboard
    - Upload screens

🧩 PHASE 5 — STUDENT MODULE + PARENT MODULE

Objective

Data consumption layer.

Student Features

- View marks
- View assignments
- View materials
- View notifications
- View fee status
- Update profile

Parent Features

- View child progress
- View attendance
- View assignments
- View fee history
- Complaint system:
    - Submit complaint
    - Attach image

Backend

- Parent-child relationship mapping

Database tables

- complaints
- parent_student_mapping

🧩 PHASE 6 — FEES MANAGEMENT SYSTEM

Objective

Financial management.

Features

- Define fee structure
- Assign fees to students

- Track payments
- Fee status (paid/pending)

Database

- fees
- payments

Frontend

- Fee dashboard
- Payment history

🧩 PHASE 7 — NOTIFICATIONS + FCM + REAL-TIME SYNC

Objective

Real-time updates and alerts.

Requirements

1. Integrate Firebase Cloud Messaging:
2. Backend:
    - Store FCM tokens
    - Send notifications:
        - Fee reminders
        - Assignments
        - Marks updates
3. Mobile:
    - Receive notifications:
        - Foreground
        - Background
        - App closed
4. Offline Sync:

    - Cache data locally
    - Sync when online
    - Notify user after sync
5. Database:
    - notifications table

🧩 PHASE 8 — CHAT + WHATSAPP + COMMUNICATION

Objective

Communication system.

Features

- In-app chat:
    - Parent ↔ Admin
    - Parent ↔ Staff
- Send message + image
- WhatsApp integration (deep link)

Database

- chats
- messages

🧩 PHASE 9 — UI/UX + THEMES + PERMISSIONS

Objective

Modern UI and device integration.

Requirements

1. UI:
    - Glassmorphism design
    - Smooth animations

    - Responsive layouts
2. Themes:
    - Dark mode
    - Light mode
3. Permissions:
    - Camera
    - Storage
    - Microphone

🧩 PHASE 10 — SECURITY + PERFORMANCE + FINALIZATION

Objective

Production readiness.

Security

- Rate limiting
- Input sanitization
- Secure headers
- SQL injection prevention

Performance

- Pagination
- Lazy loading
- Optimized queries

Session

- Persistent login
- Token refresh

Logging

- Centralized logging system

🗄️ FINAL DATABASE SCHEMA (COMPLETE)

Generate FULL SQL including:

- **users**
- **roles**
- **sessions**
- **otp_verifications**
- **students**
- **teachers**
- **parents**
- **classes**
- **assignments**
- **materials**
- **marks**
- **attendance**
- **fees**
- **payments**
- **notifications**
- **complaints**
- **chats**
- **messages**
- **parent_student_mapping**

Include:

- **Primary keys**
- **Foreign keys**
- **Indexing**

🔗 API REQUIREMENTS

Generate ALL APIs:

- **Auth APIs**
- **User management APIs**
- **Academic APIs**
- **Fee APIs**
- **Notification APIs**
- **Chat APIs**

📱 MOBILE APP REQUIREMENTS

- **Fully functional React Native app**
- **Role-based dashboards**
- **Offline support**
- **Secure storage**
- **Push notifications working in all states**

🚀 BACKEND REQUIREMENTS

- **Fully functional Node.js server**
- **Scalable architecture**
- **REST APIs**
- **FCM integration**
- **File upload system**

❗ EDGE CASES (MANDATORY)

- **Offline users receive updates later**

- **OTP resend limits**
- **Duplicate users prevention**
- **File upload size limits**
- **Unauthorized access blocking**
- **Multi-device login handling**

📜 FINAL OUTPUT REQUIREMENT

For EACH PHASE you MUST generate:

1. **Backend code**
2. **React Native code**
3. **Database schema updates**
4. **API routes**
5. **Middleware**
6. **Error handling**

✅ END OF MASTER PROMPT